Análisis y Diseño de Algoritmos I

Valeria Bengolea
Departamento de Computación
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Universidad Nacional de Río Cuarto

(notas basadas en el material adicional de "The Design & Analysis of Algoritmos" (Levitin 2003).

La Estrategia Decrease & Conquer

Escenas del Capítulo anterior...

Divide & Conquer

Una forma general de describir Divide & Conquer, es la siguiente:

- Un problema se divide en varios subproblemas del mismo tipo, idealmente del mismo tamaño
- Se resuelve cada subproblema , en general de manera recursiva.
- Las soluciones de cada subproblema se combinan para conseguir una solución al problema original.

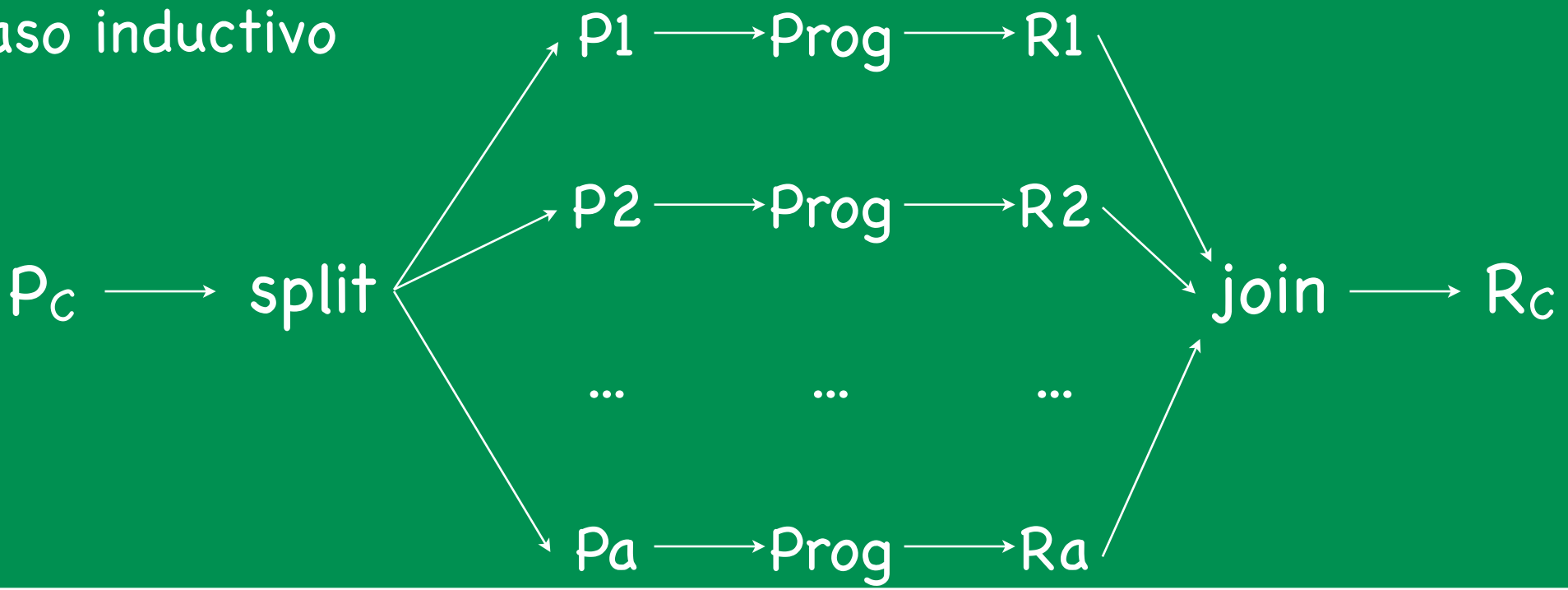
Esquema General de la Estrategia Divide & Conquer

Prog =  + 

caso base

P_B —— solución directa ——→ R_B

paso inductivo



Decrease & Conquer

- Explota la relación que existe entre una solución de una instancia de un problema y una solución de instancias más pequeñas del problema.
- Cuando se establece la relación podríamos aprovechar esa relación top down o bottom up.

Decrease & Conquer

La estrategia de diseño de algoritmos Decrease & Conquer apunta a resolver problemas algorítmicamente de la siguiente forma:

- Reducir instancias del problema a una instancia más pequeñas del mismo problema
- Resolver la instancia más pequeña
- Extender la solución de la instancia más pequeña para conseguir una solución a la instancia original

Tipos de Decrease & Conquer

De acuerdo a la forma de "decrementar" un problema, existen diferentes tipos de Decrease & Conquer

- **Decremento por una constante** (usualmente 1):
  - insertion sort / selection sort
  - algoritmos de recorrido de grafos (DFS and BFS)
  - ordenamiento topológico
  - algoritmos para generar permutaciones, subconjuntos, ...
- **Decremento por un factor constante** (usualmente la mitad)
  - búsqueda binaria
  - exponenciación por cuadrados
- **Decremento de tamaño variable**
  - Algoritmo de Euclides

Decrease & Conquer

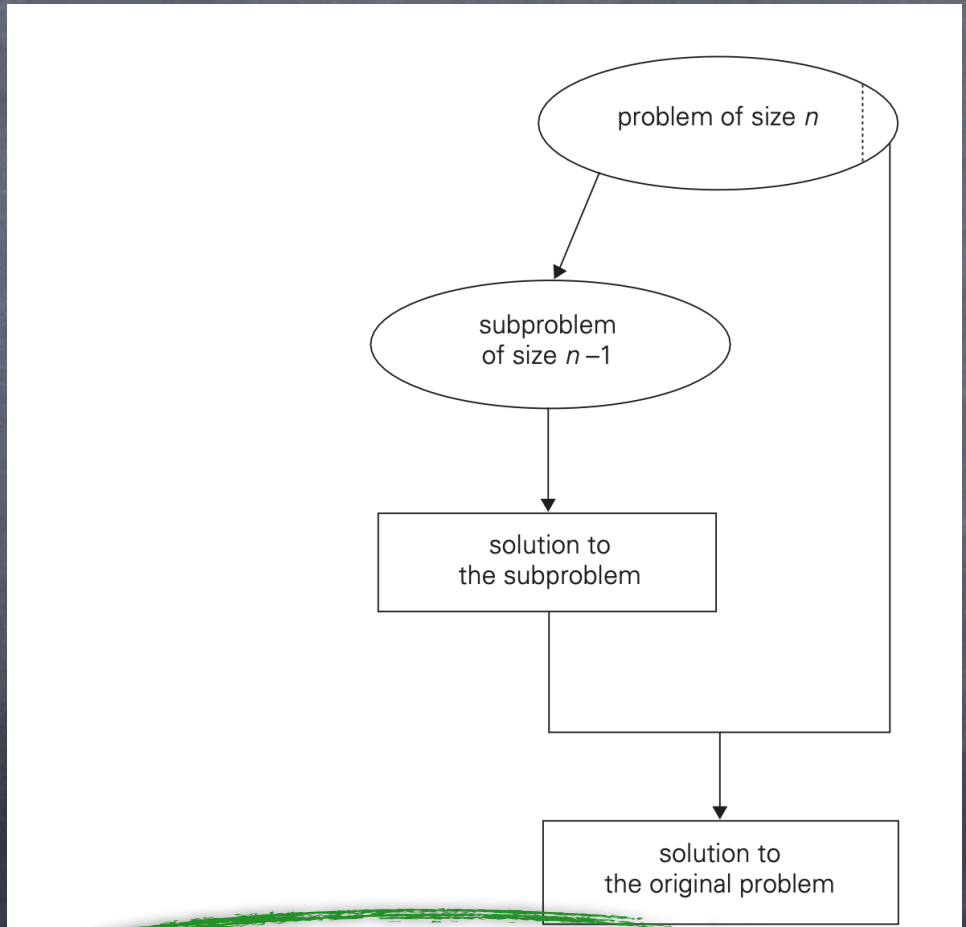


FIGURE 4.1 Decrease-(by one)-and-conquer technique.

Un ejemplo: Exponenciación

Problema: Computar a^n, donde n es un entero no negativo.

Solución Decrease & Conquer: Para computar a^n, con n >0, podemos computar a^{n-1}, y multiplicar el resultado por a.

$$a^1 = a$$
$$a^n = a^{n-1} * a$$

Recurrencia: $T(1) = 0$

$$T(n) = T(n-1) + 1 = \ldots = n - 1 \in \Theta(n)$$

Generando Permutaciones

- Problema: Generar todas las permutaciones de la lista [1,2...,n]
- Solución decrease (by one) & Conquer:
  - Generar todas las permutaciones de la lista [1...n-1]
  - Insertar n, en cada una de las n posiciones posibles de cada permutación de n-1 elementos.

Interleave x xs?

```
permutations :: [a] -> [[a]]
permutations [x] = [[x]]
permutations (x:xs) =  concatMap (interleave x) ps
                       where ps = permutations xs
```

Generando Subconjuntos

- Problema: Generar todos los subconjuntos de un conjuntos $A = \{a_1, a_2, .. a_n\}$
  - Solución decrease (by one) & Conquer:
    - Todos los subconjuntos de $A = \{a_1, a_2, .. a_n\}$ pueden dividirse en dos grupos:
    - Aquellos que no contienen a_n y aquellos que sí lo contienen.
    - Los que NO lo contienen son todos los subconjuntos de $A = \{a_1, a_2, .. a_{n-1}\}$
    - Aquellos que contienen a a_n pueden formarse agregando a_n a cada uno de los conjuntos del punto anterior.

```
subsets :: [a] -> [[a]]
subsets [] = []
subsets [x] = [[x],[]]
subsets (x:xs) = ps ++ [(x:r)| r <- ps]
                    where ps = subsets xs
```

Generando Subconjuntos

- Problema: Generar todos los subconjuntos de un conjuntos $A = \{a_1, a_2, .. a_n\}$
  - Solución decrease (by one) & Conquer:

$subsets([a_1, a_2, a_3])$ $(bottom - up)$

$\varnothing$							
$\varnothing$	$\{a_1\}$						
$\varnothing$	$\{a_1\}$	$\{a_2\}$	$\{a_1, a_2\}$				
$\varnothing$	$\{a_1\}$	$\{a_2\}$	$\{a_1, a_2\}$	$\{a_3\}$	$\{a_1, a_3\}$	$\{a_2, a_3\}$	$\{a_1, a_2, a_3\}$

upos:

-1}

  - Aquellos que contienen a a_n pueden formarse agregando a_n a cada uno de los conjuntos del punto anterior.

```
subsets :: [a] -> [[a]]
subsets [] = []
subsets [x] = [[x],[]]
subsets (x:xs) = ps ++ [(x:r)| r <- ps]
                    where ps = subsets xs
```

Generando Permutaciones

- Problema: Ge… de la lista [1,2…,n]
- Solución decr…
  - Generar to… ta [1…n-1]
  - Insertar n,… s posibles de cada perm…

permutations([1,2,3]) (*bottom* – *up*)

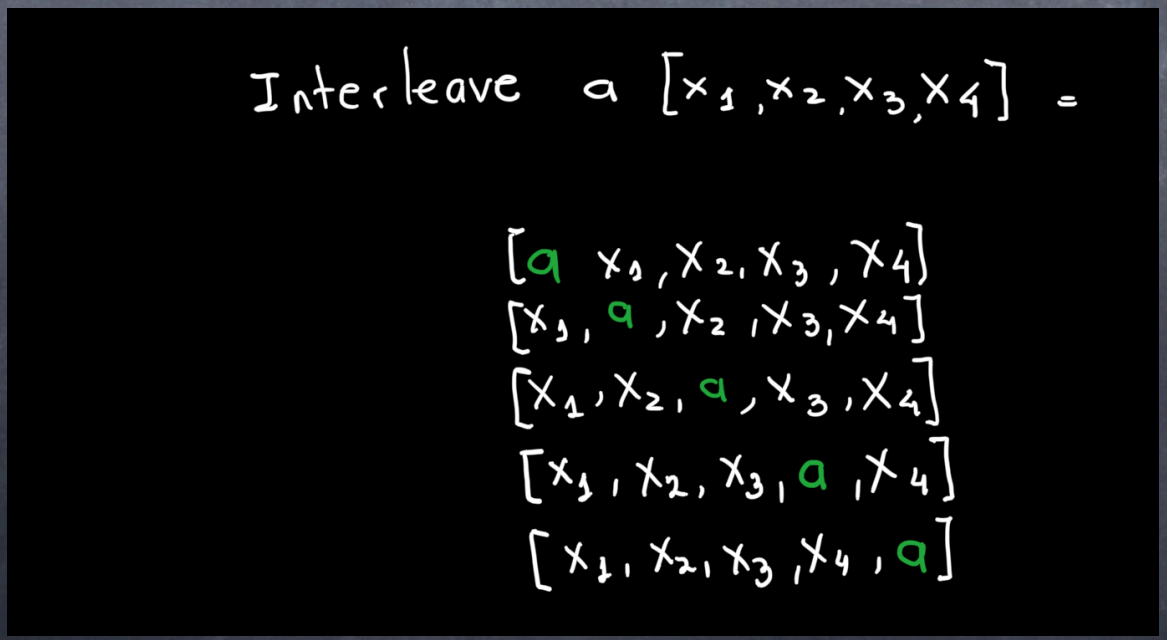
1		
12	21	
123	132	312
321	231	213

Interleave x xs?

```
permutations :: [a] -> [[a]]
permutations [x] = [[x]]
permutations (x:xs) =  concatMap (interleave x) ps
                    where ps = permutations xs
```

Otro ejemplo Decrease & Conquer...

- Problema de Interleaving en una secuencia: Dado un elemento 'x' y una secuencia, generar todas las secuencias en donde x esta intercalando en la secuencia.



Ejemplo: Ordenamiento

Para ordenar un arreglo A[0..n-1], ordenar primero A[0..n-2] y luego insertar A[n-1] en el lugar que le corresponde dentro del segmento ya ordenado A[0..n-2]

Ejemplo: ordenar 6, 4, 1, 8, 5

6 | 4 1 8 5

4 6 | 1 8 5

1 4 6 | 8 5

1 4 6 8 | 5

1 4 5 6 8

Insertin Sort

Pseudo-código de Insertion Sort (bottom up)

ALGORITHM *InsertionSort*$(A[0..n-1])$

```
//Sorts a given array by insertion sort
//Input: An array A[0..n − 1] of n orderable elements
//Output: Array A[0..n − 1] sorted in nondecreasing order
for i ← 1 to n − 1 do
    v ← A[i]
    j ← i − 1
    while j ≥ 0 and A[j] > v do
        A[j + 1] ← A[j]
        j ← j − 1
    A[j + 1] ← v
```

Operación básica

$$C_{worst}(n) = \sum_{i=1}^{n-1} \sum_{j=0}^{i-1} 1 = \sum_{i=1}^{n-1} i = \frac{(n-1)n}{2} \in \Theta(n^2)$$

Pseudo-código de Insertion Sort (Top down)

```
insertionSortRecursive(A[0..n-1], n) {

      // Base case
      if (n <= 1)
          return;

      // Sort first n-1 elements
      insertionSortRecursive(A, n-1);

      /**
       * Insert last element at its correct position in sorted array.
       */
      int last = A[n-1];
      int j = n-2;

      /** Move elements of arr[0..i-1], that are greater than key, to one
        * position ahead of their current position **/
      while (j >= 0 && A[j] > last ) {
          A[j+1] = A[j];
          j--;
      }
      array[j+1] = last;
  }
```

Algoritmos de Decremento por Factor Constante

En esta variante de Decrease & Conquer, el tamaño de instancia de problema se reduce en un mismo factor en cada caso (usualmente, 2)

Ejemplos:

- Búsqueda binaria
- Exponenciación mediante cuadrados
- Problema de la moneda falsa

Búsqueda Binaria

Master Theorem If $f(n) \in \Theta(n^d)$ where $d \geq 0$ in recurrence (5.1), then

$$T(n) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d, \\ \Theta(n^d \log n) & \text{if } a = b^d, \\ \Theta(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

Analogous results hold for the O and Ω notations, too.

```
K
//Output: An index of the array's element that is equal to K
// or –1 if there is no such element

BinarySearchRecursive(A[0..n – 1], K)
    return BinarySearchAux(A, 0, n – 1, K)

BinarySearchAux(A, l, r, K)
    if l > r return –1
    m ← ⌊(l + r)/2⌋
    if K = A[m] return m
    else if K < A[m]
        return BinarySearchAux(A, l, m – 1, K)
    else
        return BinarySearchAux(A, m + 1, r, K)
```

Podríamos aplicar el Teorema maestro?

$a = 1, b = 2, d = 0, \Theta(log_2 n)$

$$T(1) = 1,\ T(n) = T(\frac{n}{2}) + 1 = \ldots = T(\frac{n}{2^i}) + i,\ con\ i = log_2 n,\ T(n) = log_2 n + 1$$

Búsqueda Binaria

ALGORITHM *BinarySearch*$(A[0..n-1], K)$

```
//Implements nonrecursive binary search
//Input: An array A[0..n − 1] sorted in ascending order and
//       a search key K
//Output: An index of the array's element that is equal to K
//       or −1 if there is no such element
l ← 0;    r ← n − 1
while l ≤ r do
    m ← ⌊(l + r)/2⌋
    if K = A[m] return m
    else if K < A[m] r ← m − 1
    else l ← m + 1
return −1
```

Exponenciación por Cuadrados

El Problema: Computar a^n donde n es un entero no negativo

El problema se puede resolver aplicando recursivamente las siguientes fórmulas:

Para valores pares de n: $a^n = (a^{n/2})^2$ si $n > 0$ y $a^0 = 1$

Para valores impares de n: $a^n = (a^{(n-1)/2})^2 a$

Recurrencia: $M(n) = M(\lfloor n/2 \rfloor) + f(n)$, donde $f(n) = 1$,

$M(0) = 0$

Teorema Maestro: $M(n) \in \Theta(\log n)$

Exponenciación por Cuadrados

El Problema: Computar a^n donde n es un entero no negativo

El problema se puede resolver aplicando recursivamente las si

Master Theorem If $f(n) \in \Theta(n^d)$ where $d \geq 0$ in recurrence (5.1), then

$$T(n) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d, \\ \Theta(n^d \log n) & \text{if } a = b^d, \\ \Theta(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

Analogous results hold for the O and Ω notations, too.

Par = 1

Para valores impares de n: $a^n = (a^{(n-1)/2})^2 a$

Recurrencia: $M(n) = M(\lfloor n/2 \rfloor) + f(n)$, donde $f(n) = 1$ o 2,

$M(0) = 0$

Teorema Maestro: $M(n) \in \Theta(\log n)$

Multiplicación por Duplicación

El problema: Computar el producto de dos enteros positivos

Se puede resolver mediante un decremento a la mitad, usando las siguientes fórmulas:

n x (m+ 1) = (n x m) + n.

Para valores pares de n: $n * m = \frac{n}{2} * 2m$

Para valores impares de n:

$$n * m = \frac{n-1}{2} * 2m + m \quad \text{si } n > 1$$

$$= m \quad \text{si } n = 1$$

$T(1) = 1,\ T(n) = T(\frac{n}{2}) + 1$ $\Theta(log_2 n)$

Algoritmos de Decremento Variable

En esta variante de Decrease & Conquer se reduce cada instancia del problema en tamaños variables de un paso a otro

Ejemplos:

- Algoritmo de Euclides
- Búsqueda por Interpolación
- Algoritmos sobre árboles binarios de búsqueda (no balanceados)

Algoritmo de Euclides

El Algoritmo de Euclides se basa en la aplicación iterativa de la siguiente igualdad

$$\gcd(m, n) = \gcd(n, m \bmod n)$$

Ej.: gcd(80,44) = gcd(44,36) = gcd(36, 12) = gcd(12,0) = 12

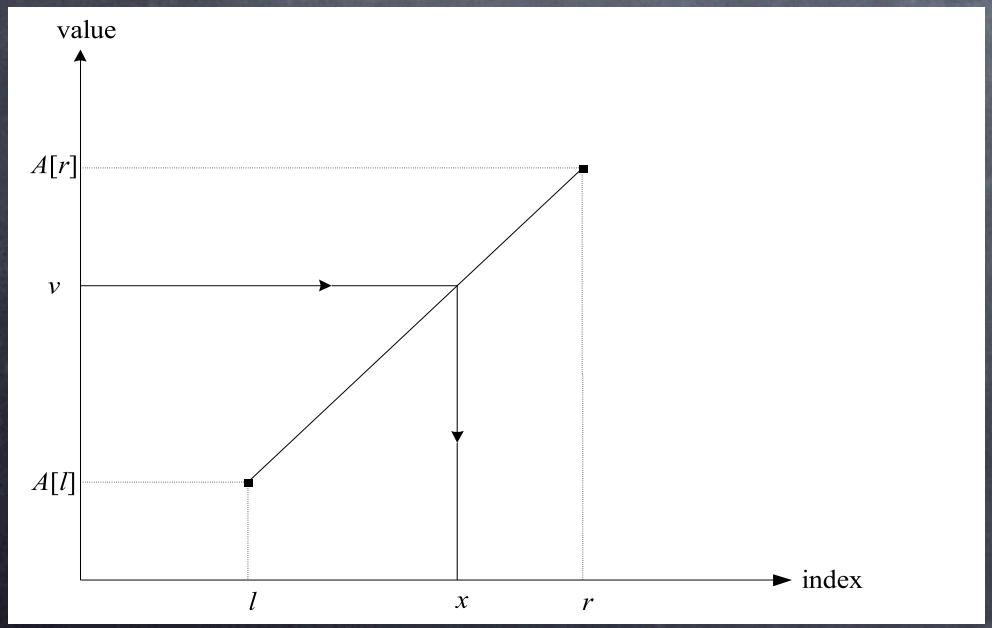
Se puede demostrar que el tamaño, medido como el segundo valor, decrece al menos a la mitad en dos iteraciones consecutivas.

Luego, $T(n) \in O(\log n)$

Búsqueda por Interpolación

Es una búsqueda en una secuencia ordenada A[l..r], similar a la búsqueda binaria, pero estimando la ubicación de la clave buscada en el segmento a explorar, usando su valor.

Más precisamente, se supone que los valores del arreglo crecen linealmente, y la ubicación de la clave v a buscar se estima como el valor de la coordenada x del punto correspondiente a v, en el segmento que une (l, A[l]) con (r, A[r]), y cuya coordenada x es v:



$$x = l + \lfloor (v - A[l]) \ * \ \frac{(r-l)}{(A[r] - A[l])} \rfloor$$

Análisis de la Búsqueda por Interpolación

- Eficiencia

  caso promedio: $C(n) < \log_2 \log_2 n + 1$

  peor caso: $C(n) = n$

- Es preferible a la búsqueda binaria sólo para búsquedas sobre espacios muy grandes, o donde las comparaciones son costosas